

CSS - Cascading Style Sheets

Formatierungssprachen für HTML-Dokumente

- HTML dient der Strukturierung und semantischen Auszeichnung
- CSS zum Formatieren/Stylen des Dokuments
- Möglichkeit für verschiedene Ausgabemedien (Bildschirm, Papier) unterschiedliche Darstellungen vorzugeben: **Responsives Design!**

Trennung von Inhalt und Design

- Optimalfall: Austausch des Designs ohne Veränderung des Dokuments
- „**inline**“-HTML-Style vermeiden, keine font-Tags, o.Ä. verwenden
 - > Besser: HTML-Elemente gruppieren und über Attribute mit Bezeichner versehen. Hierüber CSS-Klassen und Ids mit Formatierungen versehen
 - > Idee: Wiederverwendbare Stil-Elemente schaffen (Bootstrap)

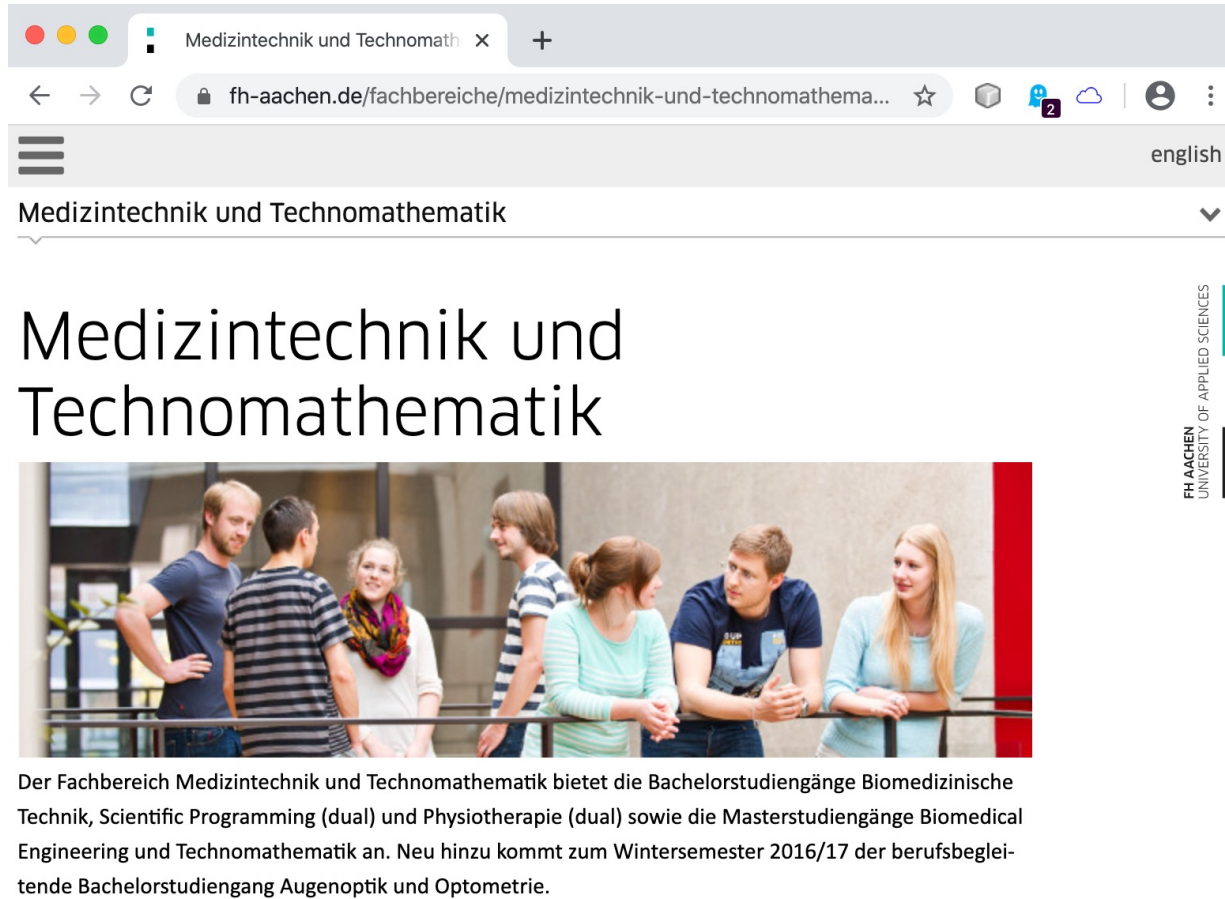
Beispiele

- Farben (Vordergrund, Hintergründe, Rahmen)
- Schriftgrößen, -farben, -arten, -stile
- Textformatierungen, Rahmen, Ränder
- Positionierung (pixelgenau!)
- ...

CSS – Wie kann ich responsives Design etablieren?



CSS oder: Wie kann ich responsives Design etablieren?



The screenshot shows a web browser window displaying the website of the Faculty of Biomedical Engineering and Technomathematics at FH Aachen. The browser's address bar shows the URL `fh-aachen.de/fachbereiche/medizintechnik-und-technomathema...`. The website has a header with a hamburger menu icon on the left and the word "english" on the right. Below the header, the text "Medizintechnik und Technomathematik" is displayed with a downward arrow. The main content area features a large heading "Medizintechnik und Technomathematik" and a photograph of a group of seven students standing outdoors and talking. Below the photo, a paragraph of text describes the faculty's offerings: "Der Fachbereich Medizintechnik und Technomathematik bietet die Bachelorstudiengänge Biomedizinische Technik, Scientific Programming (dual) und Physiotherapie (dual) sowie die Masterstudiengänge Biomedical Engineering und Technomathematik an. Neu hinzu kommt zum Wintersemester 2016/17 der berufsbegleitende Bachelorstudiengang Augenoptik und Optometrie."

Medizintechnik und Technomathematik

Medizintechnik und Technomathematik

Der Fachbereich Medizintechnik und Technomathematik bietet die Bachelorstudiengänge Biomedizinische Technik, Scientific Programming (dual) und Physiotherapie (dual) sowie die Masterstudiengänge Biomedical Engineering und Technomathematik an. Neu hinzu kommt zum Wintersemester 2016/17 der berufsbegleitende Bachelorstudiengang Augenoptik und Optometrie.

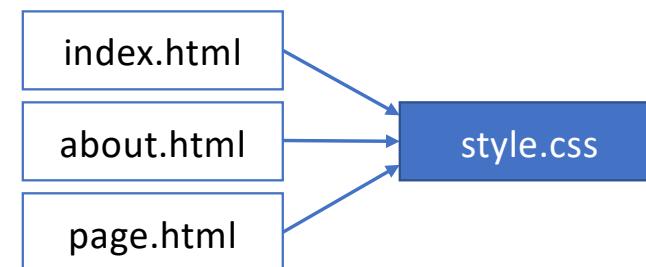
CSS

Anweisungen einbinden in das Dokument

1. Einbinden im <head> (**empfohlen!**)

```
<link rel="stylesheet" href="style.css" type="text/css">
```

- > Wiederverwenden der Formatierungen über mehrere Dokumente hinweg
- > Style-Dateien liegen hier auf dem Gleichen Server. Für Bibliotheken werden Hierfür aber spezielle Content-Delivery Networks (CDN) eingesetzt



2. style-Element im <head> codieren

```
<style type="text/css">
  h1 { font-size: 200%; }
</style>
```

3. style-Attribut (**unschön, nicht wiederverwendbar**)

```
<h1 style="font-size:200%;">Überschrift</h1>
```

Aufbau eines CSS-Dokumentes

```
Selektor {  
    Eigenschaft: Wert;  
    Eigenschaft: Wert;  
}  
Selektor1, Selektor2 {  
    Eigenschaft: Wert;  
    /* Kommentar */  
}
```

- > Selektoren sprechen eine Menge von HTML-Elementen an
- > Spezifizierte Eigenschaften werden auf diese HTML-Elemente angewendet
- > Kommentare immer so: /* Kommentar */

Beispiele (immer für alle entsprechenden Tags, wenn nicht anders angegeben)

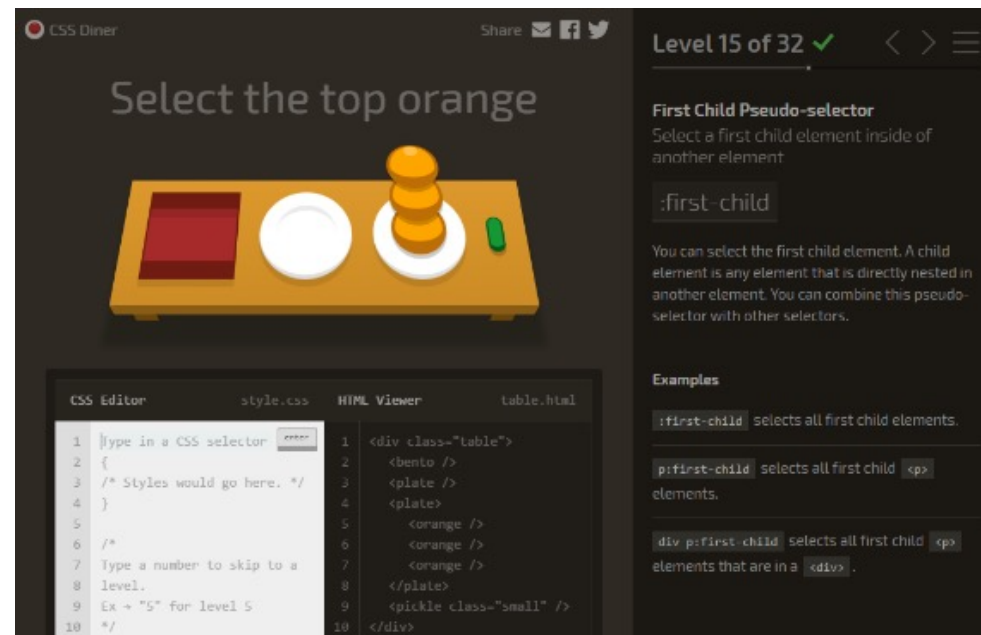
```
h1 { font-size:200%; }
p strong { ... }
p.first { ... }
p, li { ... }
input[name=pw] { ... }
.h1 { ... }
div#navi { ... }
a:hover { ... }
```

```
/* <h1>...</h1> */
/* <p><strong>...</strong></p> */
/* <p class="first">...</p> */
/* p- und li-Tags */
/* <input name="pw" .../> */
/* Alle Tags mit class="h1" */
/* <div id="navi">...</div> */
/* a-Tag beim "Mouseover" */
```

- Es existieren noch viele weitere CSS-Selektoren
 - > <https://wiki.selfhtml.org/wiki/CSS/Selektoren>
 - > https://developer.mozilla.org/en-US/docs/Web/CSS/CSS_Selectors

Spiel zum Lernen der CSS-Selektoren

- <https://flukeout.github.io/>



Werte und Maßeinheiten

- Meist besteht eine Angabe aus zwei Teilen:
Wert und Einheit
Beispiel: 20px = (20, Pixel)
- Wir sollten aber sinnvolle Einheiten wählen, die ein Responsives Design erlauben
- Relative Einheiten wählen
- Das Erscheinungsbild von Pixeln ist ja abhängig von der Auflösung (im Gegensatz zu Pt)

Relative Längenangaben

Einheit	Beschreibung	Beispiel
%	Angabe relativ zum Elternelement (bzw. Referenzelement)	<code>width: 50%;</code>
em	1em entspricht der Schriftgröße des Elementes auf das die Eigenschaft angewendet wird. Oftmals wird hier in dem Selektor auch ein Font angegeben	<code>border-width: 1em;</code>
vw, vh	Viewport Units (<i>viewport width</i> und <i>viewport height</i>) sind relativ zur Größe des Browserfensters (in %)	<code>font-size: 3vw;</code> <code>height: 100vh;</code>

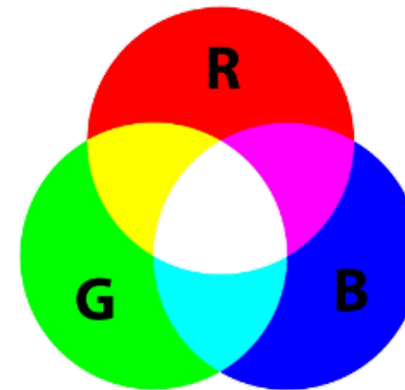
Der **Viewport** ist der **sichtbare Bereich im Browser**. Wenn ein Browserfenster verkleinert wird, so ändert sich der Wert

Weitere Einheiten:

- Für Druckmedien: cm, mm, in
- Weitere relative Einheiten: ex, ch, rem, vmin, vmax

Farben

- RGB-Farbmodell definiert eine Farbe durch die Angabe der Anteile an (rot, grün, blau)
 - > Werte im Bereich von 0 bis 255
 - > Beispiel: (255 rot, 255 grün, 0 blau) = gelb
 - > Hexadezimale Schreibweise: `#ffff00`
 - > Kurzschreibweise: `#ff0`
- RGBA erlaubt zusätzlich eine Deckkraftangabe
 - > Beispiel: `rgba(255, 255, 0, 0.5)`
 - Gelb mit 50% Deckkraft
- Ergänzend gibt es einige fest vorgegebene Farbwerte
 - > Beispiele: `red`, `blue`, `cyan`, `yellow`



Farbbeispiele (auf weißem Hintergrund)

```
.a {  
  color: red;  
}  
.b {  
  color: cyan;  
}  
.c {  
  color: #000;  
}  
.d {  
  color: #ffffff;  
}  
.e {  
  color: #f00;  
}  
.f {  
  color: #32dfa2;  
}  
.g {  
  color: rgba(255, 0, 0, 0.5);  
}
```

Text mit Klasse a

Text mit Klasse b

Text mit Klasse c

Text mit Klasse e

Text mit Klasse f

Text mit Klasse g

Text- und Schriftformatierung

```
font-family: Arial, sans-serif; /* Schriftart */
font-style: italic;             /* kursiv */
font-size: 20px;                /* Schriftgröße */
font-weight: bold;              /* fett */
color: blue;                    /* Textfarbe (blau) */
text-align: center;             /* zentrieren */
font: 20px Arial, sans-serif;   /* 2 Parameter */
```

- Seltener benötigt:
 - > text-decoration, letter-spacing, font-stretch, text-outline, text-stroke
 - > ...
 - > <https://wiki.selfhtml.org/wiki/CSS>

Rahmenformatierung

```
border-width: 10px;    /* Breite */
border-style: solid;    /* Durchgezogene Linie */
border-color: red;      /* Farbe (rot) */
border: 10px solid red; /* Zusammengefasst */
border-radius: 5px;     /* Rundung der Ecken */
```



- Aufschlüsselung der Eigenschaften in (top, right, bottom, left) möglich
> border-top, border-right, border-bottom, border-left
- **Besser relative Angaben (z.B. 1em – wie die Schriftgröße)!**

Hintergrundformatierung

```
background-color: red;           /* Farbe (rot) */
background-image: url (logo.png); /* Datei */
background-position: 10px 20px;  /* Verschiebung */
background-repeat: no-repeat;    /* Bild nicht "kacheln" */
```

- Weitere Eigenschaften:

- > -size, -attachment, -clip, -origin, ...
- > https://wiki.selfhtml.org/wiki/CSS/Tutorials/Hintergrund/Gestaltung_mit_CSS

Absolute Positionierung

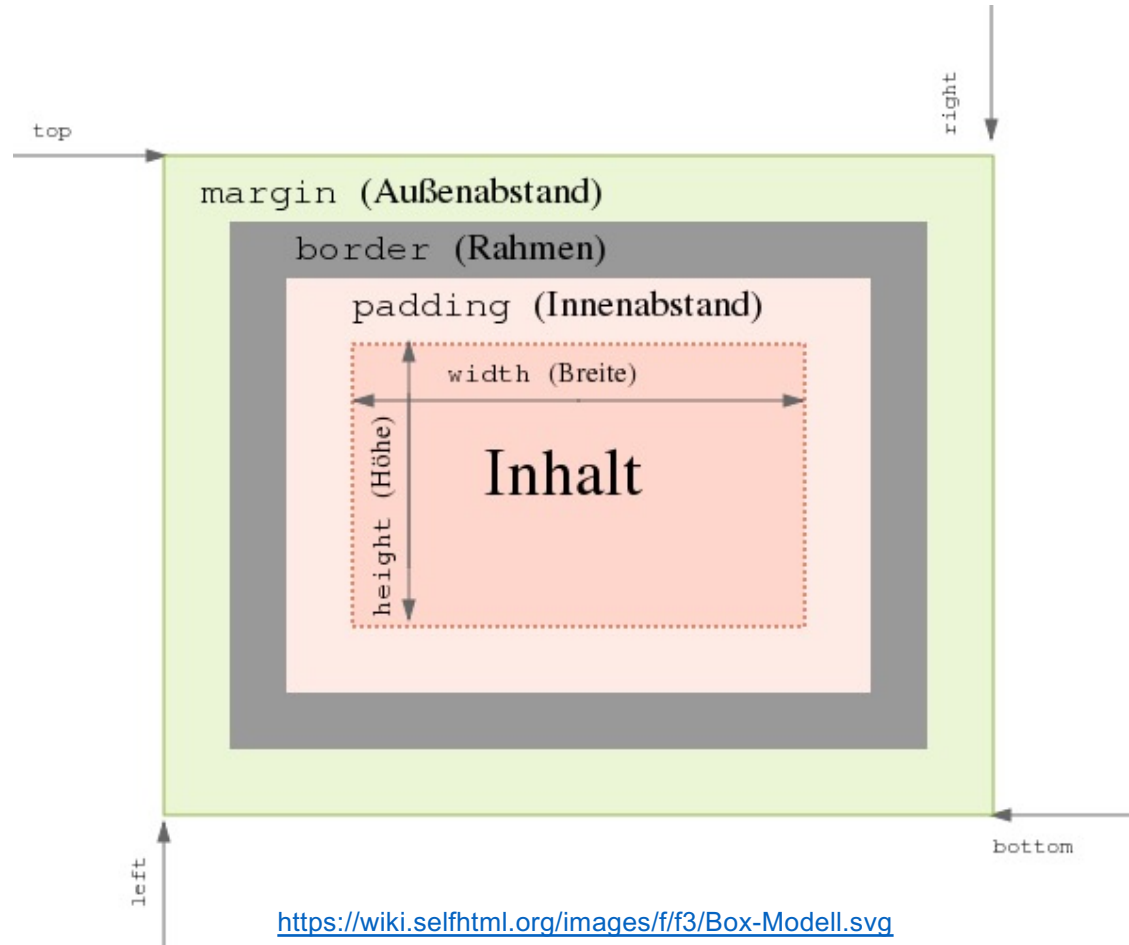
- Element exakt an der Stelle (100, 100) positionieren:

```
position: absolute;  
left: 100px;  
top: 100px;
```

> nur in seltenen Fällen notwendig, tatsächlich sogar kontraproduktiv

Formatierung einer Box

```
width: 100px;           /* Breite */  
height: 100px;          /* Höhe */  
margin: 10px;           /* Außenabstand */  
padding: 5px 10px 5px 10px; /* Innenabstand */  
display: none;          /* "Display"-Typ */
```

Formatierung einer Box

Auch hier gilt:

- Besser keine festen Werte für die Breite und Höhe sondern relative (width: 27%; padding: 1em;)
- Festlegen der Abstände über padding
- Nutzen relativer Längenmaße die sich an die Schriftgröße anpassen (em)

Box-Sizing Property

- Standardkalkulation:

`width + padding + border = actual width of an element`

`height + padding + border = actual height of an element`

- **Konsequenz:** Padding und Border beeinflussen die Größe der Box!
- Mittels box-sizing kann dies verhindert werden, hier beinhaltet die Größe der Box auch Padding und Border:

`box-sizing: border-box`

CSS

Anzeige & Positionierung

```
.div1 {  
  width: 300px;  
  height: 100px;  
  border: 1px solid blue;  
}
```

This div is smaller (width is 300px and height is 100px).

```
.div2 {  
  width: 300px;  
  height: 100px;  
  padding: 50px;  
  border: 1px solid red;  
}
```

This div is bigger (width is also 300px and height is 100px).

```
.div1 {  
  width: 300px;  
  height: 100px;  
  border: 1px solid blue;  
  box-sizing: border-box;  
}  
  
.div2 {  
  width: 300px;  
  height: 100px;  
  padding: 50px;  
  border: 1px solid red;  
  box-sizing: border-box;  
}
```

Both divs are the same size now!

Hooray!

Empfehlung: Alle Elemente so setzen

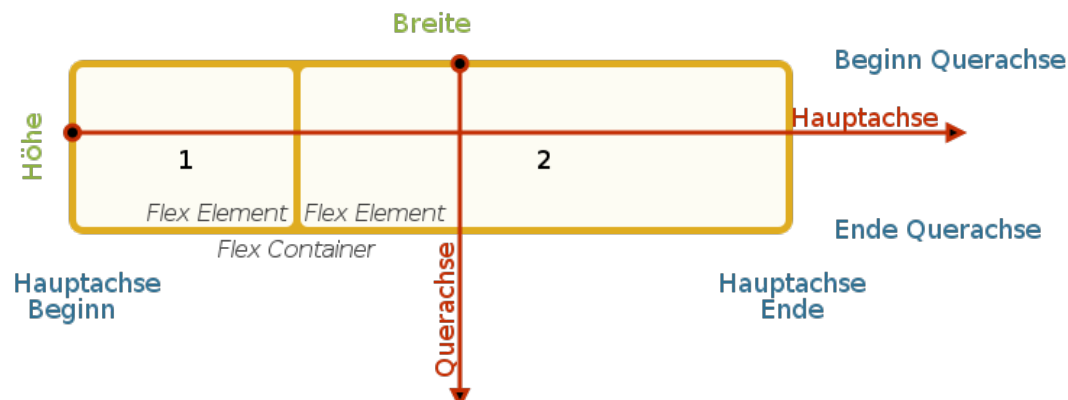
Noch besser wäre die Verwendung von em statt px!

```
* {  
  box-sizing: border-box;  
}
```

https://www.w3schools.com/css/css3_box-sizing.asp

Erst mit dem Flexbox-Container lassen sich responsive Anordnungen multipler Elemente entwerfen

- Ein Flexbox-Container beinhaltet **Flex-Items**
- Diese werden innerhalb des Flex-Containers nach den Anforderungen des Layouts angeordnet
- Flex-Items werden dabei an einer Flex-Line ausgerichtet, die ohne weitere Angabe linksbündig und horizontal ausgerichtet ist
- Wir notieren :flex um diese Eigenschaft zu starten
- **Einzelnen Items können durch nth-of-type(zahl) differenziert werden**



<https://wiki.selfhtml.org/extensions/Selfhtml/frickl.php/Beispiel:CSS-Flexbox-1.html>

https://wiki.selfhtml.org/extensions/Selfhtml/frickl.php/Beispiel:CSS-Flexbox-1.html#view_result

```
<div class="flex-container">
  <p class="flex-item">1</p>
  <p class="flex-item">2</p>
  <p class="flex-item">3</p>
  <p class="flex-item">4</p>
  <p class="flex-item">5</p>
</div>

<div class="flex-container">
  <section class="flex-item">
    <h2>Flexbox</h2>
    <p>Flexbox ist eine moderne und einfache Möglichkeit, responsive und flexible Layouts zu erstellen, ohne feste Größenangaben und weitere CSS-Einstellungen wie <code>position</code>, <code>float</code> oder <code>clear</code> nutzen zu müssen. </p>
  </section>
  <section class="flex-item">
    <h2>Was kann Flexbox?</h2>
    <p>Flexbox verzichtet auf feste Breiten, wobei sich das flexible Element an den tatsächlich verfügbaren Platz anpasst, indem es die Leerräume zwischen den Elementen gleichmäßig verteilt. </p>
  </section>
  <section class="flex-item">
    <h2>flexibler Container</h2>
    <p>Flexbox benötigt nur einen CSS-Regelsatz:</p>
    <pre><code>.flex-container {
display: flex;
}</code></pre>
  </section>
</div>
```

1 2 3 4 5

Flexbox

Flexbox ist eine moderne und einfache Möglichkeit, responsive und flexible Layouts zu erstellen, ohne feste Größenangaben und weitere CSS-Einstellungen wie `position`, `float` oder `clear` nutzen zu müssen.

Was kann Flexbox?

Flexbox verzichtet auf feste Breiten, wobei sich das flexible Element an den tatsächlich verfügbaren Platz anpasst, indem es die Leerräume zwischen den Elementen gleichmäßig verteilt.

flexibler Container

Flexbox benötigt nur einen CSS-Regelsatz:

```
.flex-container {
display: flex;
}
```

```
.flex-container {
display: flex;
}

.flex-item {
border: 1px solid;
border-radius: 0 .5em .5em;
margin: .5em;
padding: .5em;
background: #ffebe6;
}

p.flex-item {
font-weight: bold;
text-align: center;
}

.flex-item:nth-of-type(2) {
background: #fdfcf3;
}

.flex-item:nth-of-type(3) {
background: #ebf5d7;
}

.flex-item:nth-of-type(4) {
background: #e6f2f7;
}

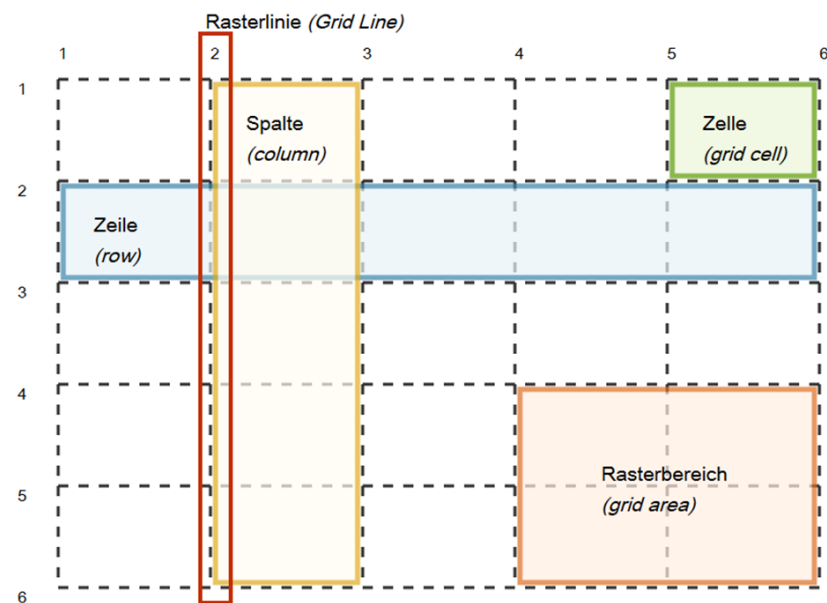
.flex-item:nth-of-type(5) {
background: hsla(277, 53%, 73%, 0.3);
}
```

https://wiki.selfhtml.org/extensions/Selfhtml/frickl.php/Beispiel:CSS-Flexbox-1.html#view_result

https://www.w3schools.com/css/css3_flexbox.asp

Mittels Flexbox können eindimensionale Strukturen verwaltet werden, CSS Grids ermöglichen zweidimensionale Layouts

- Anlegen eines Rasters
- Kindelemente werden hier ohne feste Größenangaben positioniert



<https://wiki.selfhtml.org/wiki/CSS/Tutorials/Grid/Einfuehrung>

Responsives Design wird zumeist mittels existierender Frameworks erreicht

- Bootstrap ist der populärste CSS-Framework (nicht nur CSS)
- Benötigt ab seiner Version 4 aktuellere Browser

Bootstrap verfolgt über den viewport einen Mobile-First-Ansatz

- Das Layout wird zunächst für kleine Bildschirme entworfen, um sie dann den größeren anzupassen
- `<meta name="viewport" content="width=device-width, initial-scale=1">`
- Viewport-Meta-Tag als Basis zur Behandlung des sichtbaren Bereichs

Bootstrap wird im Head-Element zumeist von einem externen Anbieter (Content Delivery Network) eingebunden und so dynamisch geladen

```
<link rel="stylesheet" href="https://maxcdn.bootstrapcdn.com/bootstrap/3.3.7/css/bootstrap.min.css" integrity="sha384-BVYiiSIFeK1dGmJRAKycuHAHRg32OmUcww7on3RYdg4Va+PmSTsz/K68vbdEjh4u" crossorigin="anonymous">
```